GPU PyFR PASS 1 – NASA Hump Reconstruction on Purdue Gilbreth

OpenAI Codex workflow inside `nasa-hump2`

May 12, 2026

1 Objective

GPU PyFR PASS 1 establishes a reproducible CUDA-backed PyFR workflow for the NASA two-dimensional wall-mounted hump case on Purdue RCAC Gilbreth. The goal of this pass is infrastructure and diagnostic honesty first: prove that PyFR can be installed in user space, submitted through Slurm on a GPU node, and advanced toward a hump reconstruction without borrowing any hidden benchmark solution data.

2 Prior OpenFOAM Baseline Summary

The repository already contains multiple OpenFOAM hump passes with official NASA Cp and Cf comparison. Two OpenFOAM checkpoints are the most relevant baselines for this GPU pass:

- PASS 4 promoted SST continuation to 650 iterations: benchmark MAE 0.165 913 674 7.
- Later plain-SST continuation to 800 iterations: benchmark MAE 0.154 738 102 8.

These baselines already include the official machine-readable NASA Cp/Cf comparison pipeline and give the PyFR workflow a physically grounded reference point.

3 PyFR Modeling Scope

PyFR PASS 1 uses `ac-navier-stokes` on the CUDA backend. This is not the same governing-equation and closure package as the existing OpenFOAM steady RANS SST baseline. The comparison is therefore intentionally framed as a cross-method reconstruction attempt rather than a claim of SST equivalence. Any mismatch or failure is documented as a modeling or implementation limitation, not hidden.

4 Gilbreth GPU Environment

The exact environment is recorded in `documentation_src/GPU_PYFR_PASS1_ENVIRONMENT.md`. The verified stack used during PASS 1 was:

- access path: `ssh gilbreth`,
- scratch workspace: `/scratch/gilbreth/prabhu56/nasa-hump2-gpu-pyfr-pass1`,
- compiler and MPI modules: `gcc/11.5.0`, `openmpi/4.1.6`,
- CUDA module: `cuda/12.6.0`,

- Python: `3.9.25`,
- PyFR: `1.15.0`,
- compute-node GPU seen in Slurm jobs: `NVIDIA A100-PCIE-40GB`.

5 Workflow and Reusable Files

PASS 1 added a full remote-execution pipeline around PyFR:

- `scripts/setup_gilbreth_pyfr_env.sh` creates the Gilbreth virtual environment and records environment details.
- `scripts/pyfr/generate_pyfr_meshes.py` builds the smoke mesh and the hump meshes with the same reconstructed lower wall and contoured upper wall used by the OpenFOAM passes.
- `scripts/pyfr/build_gpu_pyfr_pass1_case.py` generates PyFR configuration files and sampling manifests.
- `scripts/submit_pyfr_smoke_gilbreth.sh`, `scripts/submit_pyfr_medium_gilbreth.sh`, and `scripts/submit_pyfr_promoted_gilbreth.sh` provide Slurm submission wrappers.
- `scripts/fetch_gpu_pyfr_pass1_results.sh` and `scripts/postprocess_gpu_pyfr_pass1.py` pull lightweight results back into the repo and generate the PASS 1 figures and tables.

6 Mesh and Case Reconstruction

The PyFR hump geometry reuses the existing repository reconstruction choices:

- chord length 0.42 m,
- lower hump wall represented by the repo's smooth cosine-squared hump,
- mild contoured upper wall retained to approximate blockage guidance from the allowed NASA sources,
- inlet located well upstream of the hump rather than directly at the first validation station.

The PASS 1 mesh path was deliberately incremental:

1. a rectangular smoke mesh to validate the CUDA backend and Slurm path,
2. a curved structured hump mesh,
3. a structured triangular hump mesh,
4. a piecewise-linear triangular hump mesh as a more robust fallback.

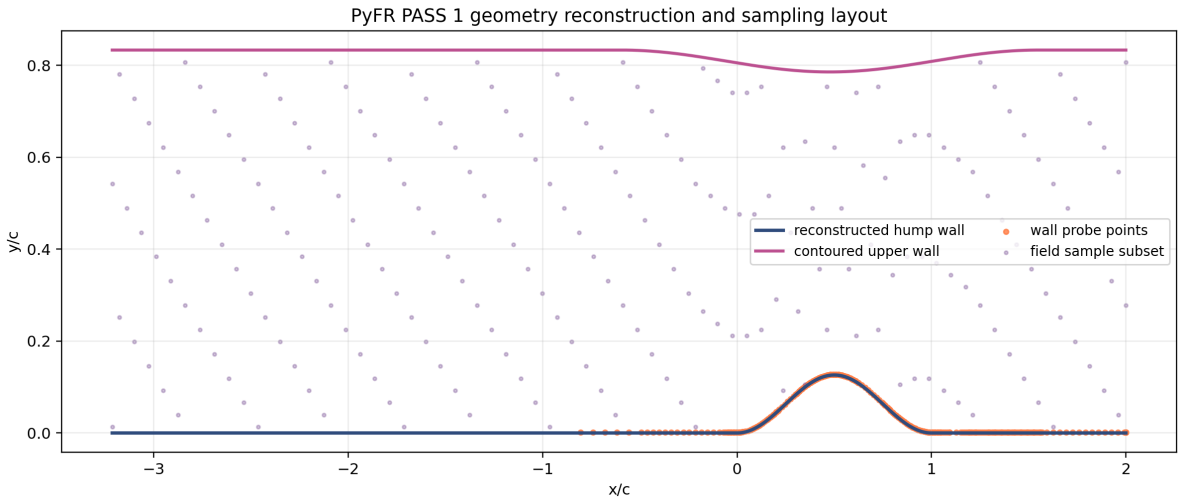

Figure 1: Geometry and sampling layout used to build the PyFR hump attempt. The lower wall and upper contour come from the repository's existing reconstruction logic, while the sample points are generated automatically from the local NASA comparison pipeline.

7 PyFR Numerical Setup

The generated PyFR configurations use:

- `ac-navier-stokes`,
- double precision,
- CUDA backend with `device-id = local-rank`,
- `mpi-type = standard`,
- dual-time integration,
- conservative first-pass polynomial orders and time-step settings,
- no-slip lower wall and slip upper wall,
- sampler plugins for wall, field, profile, and benchmark points on the hump runs.

Those settings were chosen to start from a numerically conservative baseline instead of attempting an overly aggressive production-quality run on the first GPU pass.

8 Slurm Job Workflow

All meaningful work was submitted through Slurm on the `a100-40gb` partition. The smoke test was required to succeed before escalating to the hump case. The promoted hump job was intentionally not submitted after the medium hump attempts exposed unresolved PyFR mesh/sampler issues.

9 Run Results

9.1 Outcome Table

Case	Status	Benchmark MAE	Cp MAE	Cf MAE	Notes
OF PASS4	completed	0.165914	0.171180	0.000848	Best PASS 4 continuation to 650 iterations.
OF SST800	completed	0.154738	0.164179	0.000768	Strongest existing plain-SST baseline entering PyFR PASS 1.
OF PASS5	completed	0.135890	0.153755	0.000673	Best local OpenFOAM benchmark after later continuation to 1200 iterations.
PyFR smoke	completed	–	–	–	CUDA backend smoke test completed through Slurm; no NASA hump metrics expected.
PyFR curved	failed	–	–	–	Failed during interface face-point sorting inside PyFR element/intersection setup.
PyFR tri	failed	–	–	–	Structured triangle variant still failed in interface ordering before marching.
PyFR linear tri	failed	–	–	–	Solver advanced beyond import/setup, but sampler plugin failed before evaluation and wall CSV export.

9.2 Attempt Status

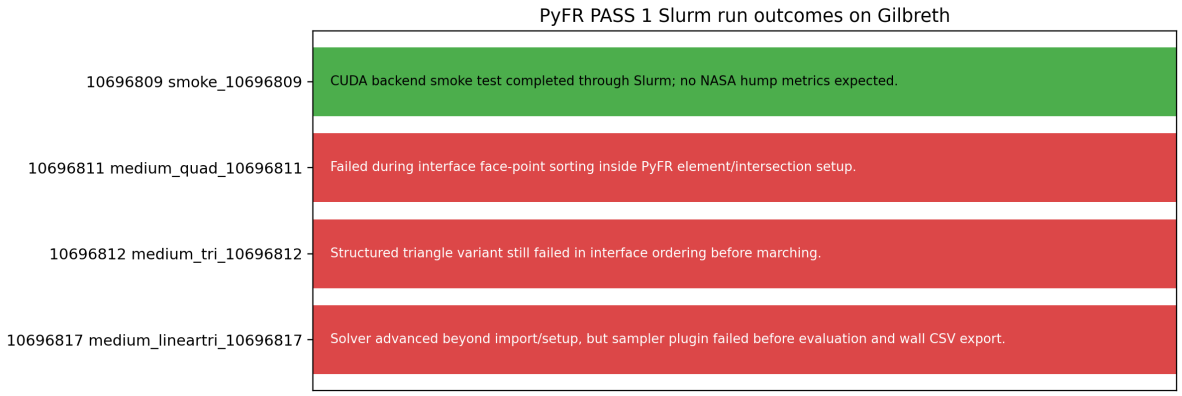

Figure 2: PASS 1 PyFR run outcomes. The smoke case completed on the GPU. The hump attempts progressively moved from mesh/interface failures to a later sampler-plugin failure after solver setup.

9.3 Smoke-Test Diagnostics

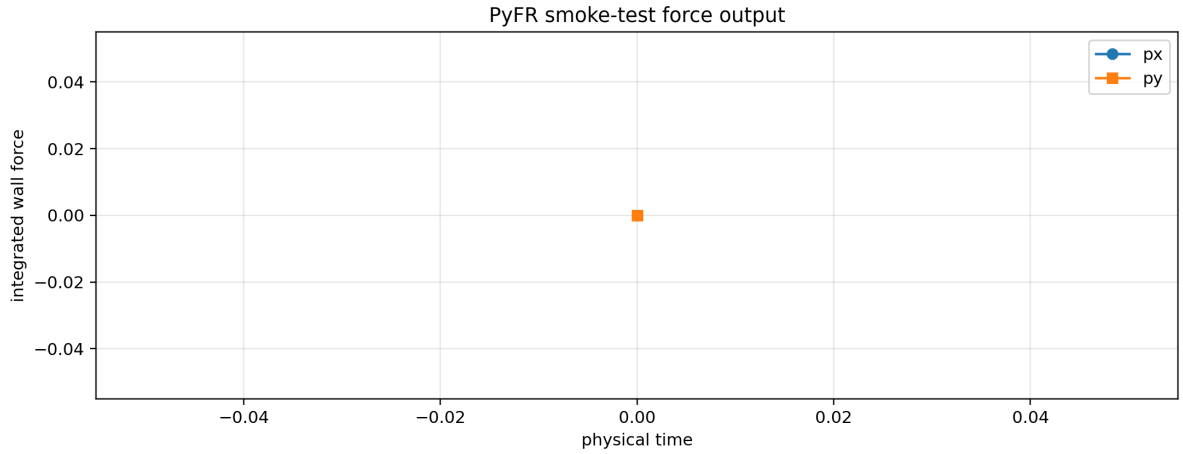

Figure 3: Smoke-test force output. The result is useful primarily as a plugin-execution sanity check rather than a physically meaningful aerodynamic signal.

10 Cp and Cf Comparison Against NASA Data

PASS 1 preserved the official machine-readable NASA Cp and Cf import path, but the hump run did not reach a state where valid PyFR wall-distribution CSV files were written. Specifically, job `10696817` advanced beyond PyFR import and solver setup on the piecewise-linear triangular hump mesh, but then failed inside the sampler plugin when refining the requested sample-point set. Because of that, PASS 1 does *not* claim a valid PyFR hump Cp or Cf curve.

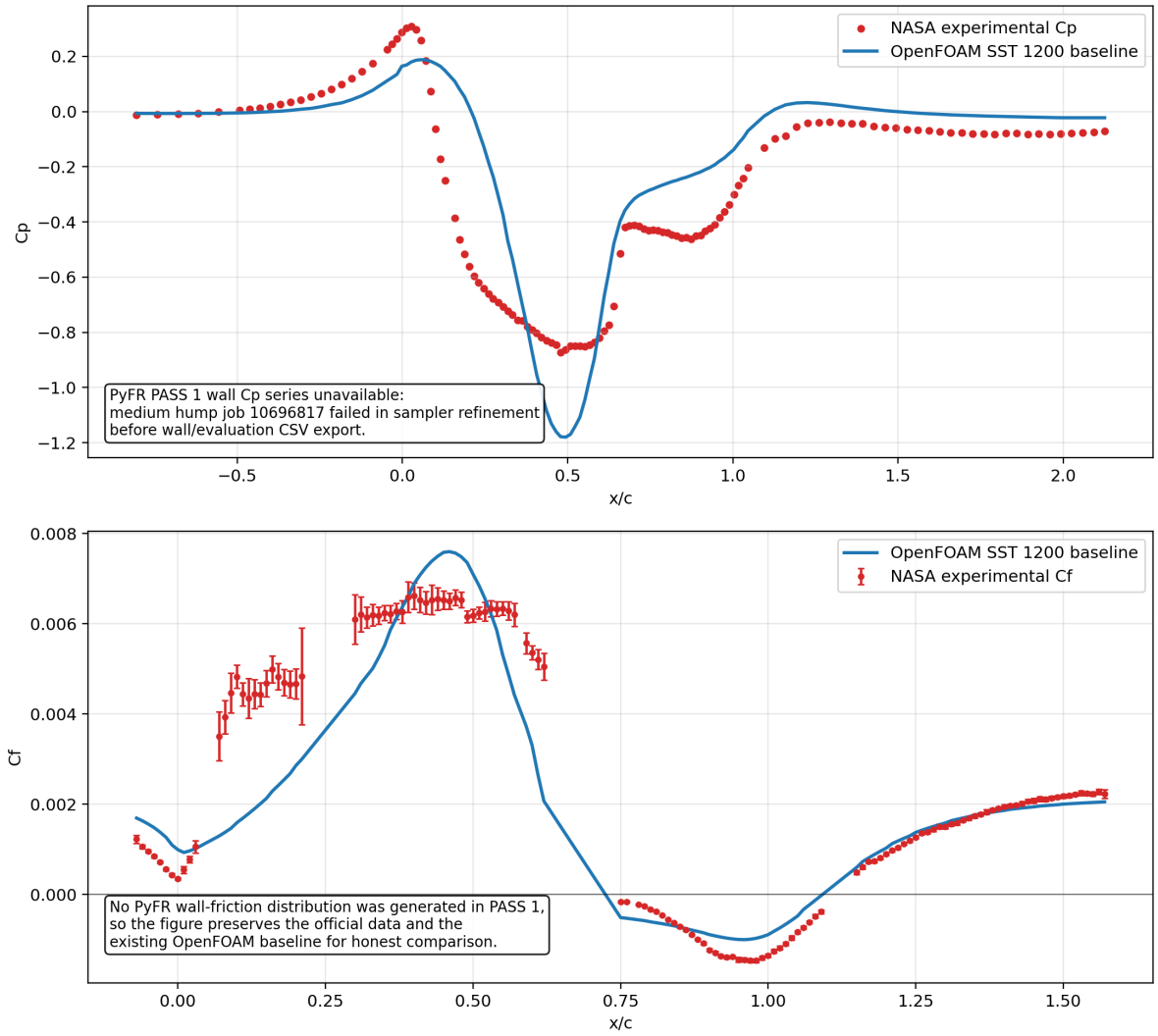

Figure 4: Official NASA wall-coefficient data together with the best existing OpenFOAM baseline already present in the repository. PASS 1 did not produce a valid PyFR wall series, so the figure keeps the comparison honest instead of fabricating a PyFR overlay.

11 Comparison Against the OpenFOAM Baseline

The strongest OpenFOAM continuation already in the repository remains the reference standard for this problem. Relative to that baseline, GPU PyFR PASS 1 achieved the following:

- successful user-space installation of PyFR on Gilbreth,
- successful Slurm GPU smoke test on an A100 node,
- successful reconstruction and import attempts for multiple hump mesh variants,
- one hump attempt that advanced into solver execution before failing in the sampler stage,
- no valid hump benchmark MAE, Cp MAE, or Cf MAE yet.

So PASS 1 is scientifically useful as a GPU-workflow baseline, but it is not yet competitive with the OpenFOAM hump benchmark.

12 Failure Modes and Limitations

The dominant PASS 1 limitations are:

- PyFR PASS 1 does not reproduce OpenFOAM SST physics; it uses `ac-navier-stokes`.
- The curved and structured hump meshes failed in PyFR face-point sorting during interface setup.
- The linearized triangular hump mesh progressed further, but the sampler plugin failed before benchmark and wall-comparison CSV export.
- Without valid hump wall/evaluation output, PASS 1 cannot honestly quote a PyFR benchmark MAE against the NASA hump.
- The smoke-test plugin outputs contained only startup-level force information, which is enough for plumbing validation but not for aerodynamic interpretation.

13 Reproducibility Commands

```
cd /Users/ashmitprabhu/github/nasa-hump2
ssh gilbreth
cd /scratch/gilbreth/prabhu56/nasa-hump2-gpu-pyfr-pass1/repo_git
bash scripts/setup_gilbreth_pyfr_env.sh
bash scripts/submit_pyfr_smoke_gilbreth.sh
bash scripts/submit_pyfr_medium_gilbreth.sh

# Back on the local machine
cd /Users/ashmitprabhu/github/nasa-hump2
bash scripts/fetch_gpu_pyfr_pass1_results.sh
python3 scripts/postprocess_gpu_pyfr_pass1.py
tectonic -X compile documentation_src/gpu_pyfr_pass1/main.tex --outdir
    documentation
```

14 Conclusion and Next-Step Recommendation

GPU PyFR PASS 1 succeeded as an infrastructure pass and failed as a full hump benchmark pass, which is still a valuable result. The most promising next step is not arbitrary tuning; it is to remove or simplify the sampler dependency so that a hump solution can complete and be exported first, then reintroduce benchmark and wall probes in a more PyFR-compatible way. A second practical path would be to revisit the hump mesh construction with an even more PyFR-friendly element/topology strategy before spending more cluster time on higher-fidelity comparisons.

References